Memory Management Mechanism — AgenticSys_v2

Snapshot: 2026-05-11. Reflects the codebase after commits *8ddce4e..fecdd3e* (Phase 1–3 of the Memory-Management PRD, plus async distiller and chart-to-trace refinements).

---

1. Why this exists — the problem being solved

The case-review system slows down across long reviewer conversations because the orchestrator's `input_history` accumulates every turn's full specialist tool outputs (5–50 KB each), feeding 100–500 KB of context back to the model by turn 8–10. Two compounding deficiencies:

1. No semantic recall across turns. Specialists had only intra-orchestrator-run memory, so they re-ran identical `summarize_trend` / `aggregate_column` queries on every follow-up.
2. Quantitative findings only as prose. Reviewers find numeric trends and concentrations easier to parse as charts than narrative numbers.

The memory-management subsystem replaces unbounded raw-context replay with structured, semantically-targeted recall, paired with a routing signal so the orchestrator favors warm specialists on follow-ups.

---

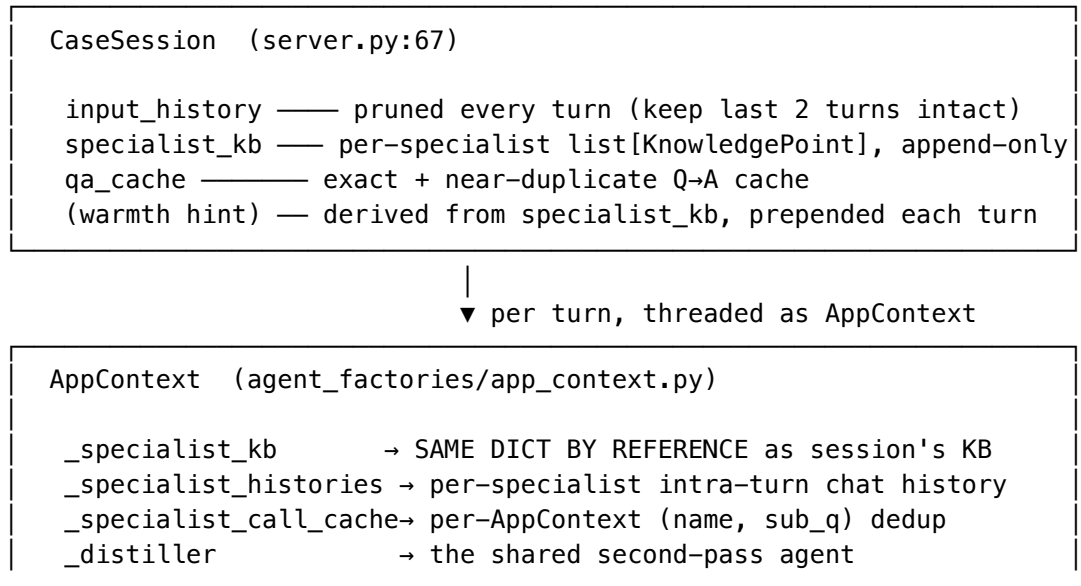
2. The core idea (one paragraph)

Each specialist's findings are distilled into atomic, quantitative `KnowledgePoints` by a second-pass agent. These KPs are stored per-specialist in a session-scoped `specialist_kb`. On the specialist's next call, only the active set (latest per topic) is fed back as a short digest preface — so the specialist sees what it already knows without replaying the raw tool output. The orchestrator's `input_history` is pruned in parallel (old turns' tool-result payloads replaced by stubs), and a one-line warmth hint prepended to the user question tells the orchestrator which specialists are already loaded with knowledge for this case.

---

3. The four memory layers

The system maintains four distinct memory layers running in parallel, all scoped to a `CaseSession` and surviving across reviewer turns within that session.



```
│   _pending_distillers   → fire-and-forget task handles              │
│   _turn_id              → tagged onto each new KP                   │
└─────────────────────────────────────────────────────────────────────┘
```

3.1 Orchestrator **`input_history`** — bounded conversational memory

- Field: `CaseSession.input_history` (`server.py:80`).
- Behavior: each turn appends user message + tool calls + tool outputs + final answer; replayed verbatim into `Runner.run_streamed` on the next turn.
- Bounding: `_prune_input_history` (`server.py:238`) runs after each turn.
  - Identifies turn boundaries by `{"role": "user", ...}` entries.
  - Keeps the last 2 reviewer turns intact (`_INPUT_HISTORY_KEEP_RECENT_TURNS`).
  - In older turns, replaces `function_call_output.output` with the stub: `> "(elided — earlier-turn specialist output; see the specialist's KB digest, which is prepended to each new sub-question.)"`
  - Function-call records themselves are preserved → orchestrator still sees that a tool ran, just not the heavy payload.
- Authoritative replay path for elided findings is the KB digest, not the history.

3.2 Per-specialist Knowledge Base — cross-turn semantic memory

- Field: `CaseSession.specialist_kb: dict[str, list[KnowledgePoint dict]]` (`server.py:97`).
- Shared by reference into each turn's `AppContext._specialist_kb` so the redacting_tool wrapper's writes persist back to the session automatically.

`KnowledgePoint` schema (`models/types.py:58`):

field	type	purpose
`topic`	`str` (snake_case slug)	grouping key for supersession (latest per topic wins)
`claim`	`str` (1 sentence)	numbers + named entities + time window; faithfully extracted
`numbers`	`list[dict]`	data series behind the claim (trends, breakdowns, breaches)
`viz`	`dict | None`	optional `{kind, x_field, y_fields}` chart spec
`source_call`	`str`	the tool invocation that produced the data
`captured_at_turn`	`str | None`	short hex turn id — chronological audit
`confidence`	`high|medium|low`	reflects the specialist's hedging

Supersession model:

- The list is append-only — never mutated, never deleted (audit trail).
- `_active_kps` in `redacting_tool.py:36` iterates and keeps the last-seen entry per topic; this is what the specialist sees on its next call.
- Older entries with the same topic stay in the list so the logger can reconstruct what was believed at any point in the session.

Read path (`redacting_tool.py:294`):

- Only on the first call to a specialist within a turn, `_format_kb_digest` prepends a one-line-per-active-KP preface:

```
[YOUR KNOWLEDGE BASE — facts established earlier this session.
 Refer to these BEFORE re-running queries; only re-query when the new
 question goes beyond what's recorded here, or when a value needs verification.]
```

```
- **monthly_spend_trend** [high]: Spend rose from $300 (2024-11) to $1,100 (2025-
03)…  _via `summarize_trend('spends',…)`_
- **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend ($642K of $1.
```

- Intra-turn follow-ups inherit the digest via the per-specialist conversation history (`_specialist_histories`), so re-prepending would duplicate.

Lifecycle: cleared by `/rewind` alongside `input_history` and `qa_cache` so a session reset wipes everything.

3.3 Async distiller — the KB writer

- Agent: `agent_factories/distiller_agent.py` — stateless, structured output (`DistillerOutput` → `knowledge_points[]`), one shared instance per orchestrator.
- Built once in `Orchestrator.__init__` (`orchestrator/orchestrator.py:61`) and exposed on `AppContext._distiller`.

Why a separate second pass instead of inline emission:

- Distillation is a different cognitive task than analysis. Asking the specialist to do both reliably bloats its prompt and degrades both.
- A narrowly-scoped agent with a strict output schema is more faithful (less paraphrasing) and cheaper to iterate.
- Failures degrade gracefully to "no KB update this turn" — the specialist's answer is unaffected.

Strict extraction rules (from the distiller prompt):

- Faithful extraction only — every claim grounded directly in the SpecialistOutput; hedges preserved.
- Atomic — one quantitative fact per point; a 12-month trend series is ONE point, not 12.
- Quantitative bias — prefer numbers, named entities, comparisons; skip pure-narrative claims.
- Skip data-absence (already in SpecialistOutput's `data_gaps`).

Fire-and-forget scheduling (`redacting_tool.py:385`):

After a specialist successfully returns:

```
task = asyncio.create_task(
    _distill_and_persist(app_ctx, name, redacted_in, result.final_output),
    name=f"distill-{name}",
)
app_ctx._pending_distillers.append(task)
return payload   # orchestrator gets answer immediately
```

- Orchestrator receives the specialist's payload without paying the distiller round-trip on the critical path.
- Server.py awaits all pending tasks at end-of-turn (`server.py:1014`, 60s budget) so the KB is fully populated before the next turn's warmth digest is built.
- Per-distiller timeout: 30s (`_DISTILLER_TIMEOUT_S`). On timeout / error, logged as `distiller_failed`; the specialist answer is unaffected.

Chart rendering side-effect:

When a distilled KP carries `viz` + non-empty `numbers`, `_distill_and_persist` also:

- Calls `kp_to_vega_spec(kp_dict)` and stores the Vega-Lite v5 spec on the KP (`kp_dict["vega_spec"] = spec`).
- Calls `render_chart(kp_dict, charts_dir, turn_id=…)` to write `reports/<case_id>/charts/<turn_id>-<topic>.png`; stores the relative URL on `kp_dict["image_path"]`.
- Failures (matplotlib error, missing fields) are logged but never raised — KP still lands in the KB, just without a chart.

Charts surface in the reasoning-trace SSE panel (not inline in the chat answer) via `_collect_turn_charts` (`server.py:323`).

3.4 Warmth hint — routing signal for the orchestrator

- Built by `_format_kb_warmth_hint` (`server.py:298`).
- Format: `[KB-warmth: spend_payments (3 KPs), modeling (5 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]`
- Sorted by descending KP count; specialists with zero KPs are omitted (no "(0 KPs)" noise).
- Prepended to the redacted user question on every turn after the first (`server.py:684`):

  ```
  framed_question = f"{warmth_hint}\n\n{verdict.redacted_question}"
  ```

- The `team_construction` skill is updated to treat this as a primary follow-up routing signal: "favor reusing warm specialists unless the question's domain has clearly shifted."
- No hard skip of team construction — the orchestrator retains full LLM judgment; the hint is a signal, not a programmatic bypass.
- Logged as `kb_warmth_hint_emitted` with `turn_id`, `warm_specialists`, `hint_length` (no PII, structural counts only).

---

4. Supporting caches

4.1 Per-specialist intra-turn history

- `AppContext._specialist_histories: dict[str, list]` (`app_context.py:22`).
- Updated by `redacting_tool` after each specialist run with `result.to_input_list()`.
- Lets a follow-up tool call to the same specialist within the same AppContext see what was already asked / answered, instead of starting fresh.
- Resets per-AppContext (i.e. per-turn).

4.2 Per-specialist call dedup

- `AppContext._specialist_call_cache: dict[tuple[name, normalized_subq], str]` (`redacting_tool.py:270`).
- Same `(specialist, normalized_sub_question)` within the same context returns the cached payload rather than re-running.
- Caps cost when the orchestrator (especially in safechain mode where parallel-tool-call semantics aren't native) emits the same call multiple times in one turn with trivial wording variations.
- `_normalize_subq` collapses whitespace + lowercases.

4.3 Session QA cache

- `CaseSession.qa_cache: dict` (`server.py:89`).
- Keyed by `_normalize_q(redacted_question)`; value carries the cached FinalAnswer fields.
- Exact-match on the redacted reviewer question → skips the orchestrator entirely on repeats.
- Near-duplicate path (`server.py:589`): `ScreenVerdict.near_duplicate_of` (computed by the relevance_check skill on subject + time-range + scope) re-keys into the cache for fuzzy matches.
- Logged as `qa_cache_hit` / `qa_cache_hit_near_duplicate`.

---

5. End-to-end flow per turn

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 1. Server receives user question                                         │
│    • screen + redact via ChatAgent                                       │
```

```
│    • near-duplicate check against qa_cache → replay if hit                  │
└─────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│ 2. Build framed question                                                    │
│    • _format_kb_warmth_hint(sess.specialist_kb)                             │
│    • framed = f"{warmth_hint}\n\n{redacted_question}"                       │
│    • run_input = sess.input_history + [{"role":"user", framed}]             │
└─────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│ 3. Orchestrator routes; specialists called via redacting_tool               │
│    For each specialist call:                                                │
│      a. dedup-cache hit? → return cached payload, done                      │
│      b. first call this turn? → prepend KB digest (active KPs)              │
│      c. prior intra-turn history? → prepend that instead                    │
│      d. Runner.run(inner, run_input, max_turns=25, timeout=240s)            │
│      e. redact_payload(result.final_output) → orchestrator                  │
│      f. schedule fire-and-forget distiller task                             │
│      g. save updated history to _specialist_histories                       │
└─────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│ 4. Orchestrator emits FinalAnswer                                           │
└─────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│ 5. End-of-turn server bookkeeping                                           │
│    • drain _pending_distillers (60s budget)                                 │
│      → KB now reflects this turn's KPs (incl. PNG + vega_spec)              │
│    • _collect_turn_charts → emit `chart` SSE events                         │
│    • _prune_input_history → stub old tool outputs                           │
│    • write qa_cache entry keyed on this question                            │
└─────────────────────────────────────────────────────────────────────────────┘
```

---

6. Worked example — three reviewer turns on one case

This walks through what happens in `CaseSession` state across three back-to-back turns on case `C-00042`, showing where each memory layer kicks in.

Turn 1 — cold start

Reviewer asks: “What’s the customer’s spending pattern over the past 6 months?”

Pre-run state:

```
sess.input_history     == []
sess.specialist_kb     == {}
```

`sess.qa_cache == {}`

Framed question (no warmth hint — KB is empty):

`What's the customer's spending pattern over the past 6 months?`

Orchestrator routes → calls `spend_payments`.

Specialist sees (no KB digest, no prior history — bare sub-question):

`What is the customer's spending pattern over the past 6 months?`

Specialist runs `summarize_trend('spends', 'Amount', 'Date', period='month', op='sum')` and `aggregate_column('spends', 'Merchant', 'Amount', op='sum')`, then returns a `SpecialistOutput`.

Fire-and-forget distiller kicks off after the redacted payload is returned to the orchestrator. It extracts:


```
[
  {
    "topic": "monthly_spend_trend",
    "claim": "Spend rose from $300 (2024-11) to $1,100 (2025-03), a 3.7× increase peaking in 202
    "numbers": [
      {"period": "2024-11", "value": 300},
      {"period": "2024-12", "value": 250},
      {"period": "2025-01", "value": 480},
      {"period": "2025-02", "value": 920},
      {"period": "2025-03", "value": 1100},
      {"period": "2025-04", "value": 760}
    ],
    "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
    "source_call": "summarize_trend('spends','Amount','Date',period='month',op='sum')",
    "captured_at_turn": "a3f7c1e9d2b4",
    "confidence": "high"
  },
  {
    "topic": "top_merchants_by_sum",
    "claim": "S BERTRAM accounts for 38% of recurring spend ($642K of $1.69M total).",
    "numbers": [
      {"group": "S BERTRAM", "value": 642000},
      {"group": "Other",     "value": 1052000}
    ],
    "viz": null,
    "source_call": "aggregate_column('spends','Merchant','Amount',op='sum')",
    "captured_at_turn": "a3f7c1e9d2b4",
    "confidence": "medium"
  }
]
```


The first KP has `viz.kind=trend` with 6 numbers → renderer writes `reports/C-00042/charts/a3f7c1e9d2b4-monthly_spend_trend.png` and attaches the Vega-Lite spec.

End-of-turn drain awaits the distiller task. Post-turn state:

```
sess.specialist_kb == {
    "spend_payments": [
        { … monthly_spend_trend KP, with image_path + vega_spec … },
        { … top_merchants_by_sum KP … },
    ]
}
sess.input_history == [<6 items: user msg + tool calls + outputs + final answer>]
```

```
sess.qa_cache      == {"what's the customer's spending pattern over the past 6 months":
                       <FinalAnswer payload>}
```

A `chart` SSE event is emitted for the trend chart; reviewer sees it in the reasoning-trace panel.

Turn 2 — warm follow-up, same domain

Reviewer asks: "Did the spending peak coincide with any payment returns?"

Pre-run state: KB has 2 KPs under `spend_payments`; the QA cache holds turn 1.

Warmth hint built by **`_format_kb_warmth_hint`**:

```
[KB-warmth: spend_payments (2 KPs). Strongly consider reusing warm specialists for in-
domain follow-ups.]
```

Framed question (warmth hint prepended):

```
[KB-warmth: spend_payments (2 KPs). Strongly consider reusing warm specialists for in-
domain follow-ups.]

Did the spending peak coincide with any payment returns?
```

Orchestrator reads the warmth hint, applies the `team_construction` rule, and reuses `spend_payments` rather than building a fresh team.

Specialist sees on its first call this turn (KB digest preface — `_active_kps` returns latest-per-topic):

```
[YOUR KNOWLEDGE BASE — facts established earlier this session.
 Refer to these BEFORE re-running queries; only re-query when the new
 question goes beyond what's recorded here, or when a value needs verification.]

- **monthly_spend_trend** [high]: Spend rose from $300 (2024-11) to $1,100 (2025-
03), a 3.7× increase peaking in 2025-Q1. _via `summarize_trend('spends','Amount','Date',period='mo
- **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend ($642K of $1.69M t

--- New question ---
Did the spending peak coincide with any payment returns?
```

The specialist now knows the peak is 2025-Q1 without re-running **`summarize_trend('spends')`**. It runs only the new call:

```
filter_rows('payments', where='status == "returned"', group_by='month')
```

It returns a SpecialistOutput referencing the existing peak and the new returns timeline.

Distiller adds one new KP:


```
{
  "topic": "payment_returns_timeline",
  "claim": "Payment returns spiked from 0 (2024-Q4) to 4 (2025-Q1) coinciding with the spend pea
  "numbers": [
    {"period": "2024-Q4", "value": 0},
    {"period": "2025-Q1", "value": 4},
    {"period": "2025-Q2", "value": 1}
  ],
  "viz": null,
  "source_call": "filter_rows('payments', where='status == \"returned\"', group_by='month')",
  "captured_at_turn": "b7e2d4f8a1c6",
  "confidence": "high"
}
```

(numbers has only 3 entries — distiller's rule says viz only when ≥ 4, so no chart this turn.)

Post-turn state:

```
sess.specialist_kb == {
    "spend_payments": [
        monthly_spend_trend         # turn 1
        top_merchants_by_sum,       # turn 1
        payment_returns_timeline,   # turn 2 ← new
    ]
}
```

input_history now has both turns; pruning hasn't triggered yet (keep_recent_turns=2).

Turn 3 — cross-domain question, supersession, history pruning

Reviewer asks: "How does that align with their FICO trajectory? Also, what does spending look like if we extend to 12 months?"

Warmth hint:

```
[KB-warmth: spend_payments (3 KPs). Strongly consider reusing warm specialists for in-domain follow-ups.]
```

Orchestrator decides this needs bureau (cold) for FICO + spend_payments (warm) for the extended trend. Because two domain specialists run, general_specialist is also invoked (cross-domain review protocol).

bureau specialist is called first. No KB digest (its KB is empty). It runs FICO history queries and returns. Distiller extracts:

```
{
  "topic": "fico_trajectory",
  "claim": "FICO declined from 712 (2024-10) to 648 (2025-04), a 64-point drop steepest in 2025-
  "numbers": [
    {"period": "2024-10", "value": 712}, {"period": "2024-11", "value": 708},
    {"period": "2024-12", "value": 695}, {"period": "2025-01", "value": 671},
    {"period": "2025-02", "value": 658}, {"period": "2025-03", "value": 651},
    {"period": "2025-04", "value": 648}
  ],
  "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
  "source_call": "summarize_trend('bureau','FicoScore','Date',period='month',op='last')",
  "captured_at_turn": "c8a3f2e5d917",
  "confidence": "high"
}
```

numbers has 7 entries + viz is set → renderer writes the chart PNG.

spend_payments specialist is called for the 12-month extension. It sees the digest (now 3 KPs) and recognizes that monthly_spend_trend only covered 6 months. It re-queries with a wider window and returns a new SpecialistOutput.

Distiller extracts a NEW **monthly_spend_trend** KP (same topic, wider window). The KB list grows to 4 entries under spend_payments — but _active_kps will hide the older 6-month version on the next call:

```
# After this turn's distillation:
sess.specialist_kb["spend_payments"] == [
    monthly_spend_trend_v1,    # turn 1 — kept for audit, hidden from digest
    top_merchants_by_sum,      # turn 1
    payment_returns_timeline,  # turn 2
    monthly_spend_trend_v2,    # turn 3 — supersedes v1 in the active view
```

]
```
# _active_kps(...) returns [top_merchants_by_sum, payment_returns_timeline, monthly_spend_trend_
```

`input_history` pruning (this is now the third reviewer turn → turn 1 ages out of the "keep recent" window):

- All `function_call_output.output` strings from turn 1 are replaced with the elision stub: `>"(elided — earlier-turn specialist output; see the specialist's KB digest, which is prepended to each new sub-question.)"`
- Function-call records themselves stay (orchestrator still sees what tools ran).
- Net effect: maybe 40 KB of raw SpecialistOutput JSON replaced by ~120 bytes of stub × N calls.

`input_history_pruned` event fires with `bytes_saved` reported.

Two charts are emitted as SSE events this turn (fico_trajectory + monthly_spend_trend_v2); the reviewer sees both in the reasoning-trace panel.

What the example demonstrates

layer	shown by
KB digest preface	Turn 2: specialist knew the peak was Q1 without re-querying.
Supersession	Turn 3: `monthly_spend_trend_v2` shadows v1 in the active digest; v1 stays in the list for audit.
Warmth hint as routing input	Turn 2: orchestrator reused the warm specialist; Turn 3: still considered both, picked correctly per question.
Fire-and-forget distiller	Specialist payloads returned to orchestrator immediately; KB updates landed at end-of-turn drain.
Chart rendering side-effect	Turns 1 + 3 wrote chart PNGs + Vega-Lite specs; Turn 2 didn't (only 3-point series).
`input_history` pruning	Turn 3: turn 1's heavy tool outputs replaced by stubs.
QA cache	If the reviewer re-asks the Turn 1 question verbatim, the orchestrator is skipped entirely.
Dedup cache	Would kick in if the orchestrator emitted two near-identical sub-questions to `spend_payments` in one turn.

---

7. Design trade-offs (the "why" behind the shape)

decision	rationale
Second-pass distillation instead of inline KP emission	Specialist + distiller are different cognitive tasks; combining them bloats prompts and degrades faithfulness. Separate agent is also cheaper to iterate.
Fire-and-forget distiller	The orchestrator should not pay the distiller round-trip on the critical path. The KB only needs to be fresh for the next turn, so end-of-turn drain is sufficient.
Append-only KB with implicit supersession	Audit trail is preserved (operators can reconstruct what was believed at any turn) while the active view stays small.
Short digest preface, not full KB replay	The job is to prevent re-querying, not to replay every detail. The specialist can still re-query when verification is needed.
Warmth hint as soft signal, not hard short-circuit	Keeps team construction LLM-driven so the orchestrator can override on clear domain shifts. Programmatic bypass would be a separate exact-near-duplicate fast-path.

decision	rationale
`input_history` pruning preserves call records	Orchestrator still sees that a tool was invoked (avoids spurious re-calls), but elides the heavy payload (saves tokens).
Charts to reasoning-trace panel, not chat answer	Keeps the chat clean (text only); reviewers get click-to-open access tied to the specific finding. Vega-Lite spec is preserved for future interactive UIs.
No RAG over KB	Digest is passed verbatim; embedding-based retrieval is deferred until KBs grow past ~50 entries per specialist (not observed in current sessions).
Domain specialists see only their own KB	Cross-domain sharing (e.g. through `general_specialist`) is intentionally deferred until use cases firm up.

---

8. Observability hooks

The EventLogger emits the following memory-related events:

event	trigger	payload (structural only — no PII)
`kb_warmth_hint_emitted`	non-empty hint built before a turn	`turn_id`, `warm_specialists[{name, n_kps}]`, `hint_length`
`distiller_kps_added`	distiller produced ≥ 1 KP	`specialist`, `n_added`, `kb_size_now`, `topics[]`, `n_with_charts`
`distiller_failed`	distiller run errored or timed out	`specialist`, `error_type`, `error_message` (truncated)
`distiller_drain_timeout`	end-of-turn drain exceeded 60s	`turn_id`, `n_pending`
`distiller_outer_failure`	task creation itself failed (rare)	`specialist`, `error_type`, `error_message`
`input_history_pruned`	history pruning produced any elisions	counts of items elided, bytes saved
`specialist_call_deduped`	dedup cache served a call	`specialist`, `sub_question_norm`
`qa_cache_hit` / `qa_cache_hit_near_duplicate`	session-level QA cache served the answer	`turn_id`, `cached_question` (redacted)
`qa_cache_store`	new answer cached after a turn	structural counts

---

9. Known limits / explicit non-goals

- No interactive client-side charts — Phase 2 ships static PNG + Vega-Lite spec; interactive UI is future work.
- No hard-skip routing — Phase 3 deliberately keeps team construction as an LLM decision.
- No RAG / embedding retrieval over KB — deferred until KBs grow past ~50 entries.
- No cross-specialist KB sharing (except future `general_specialist`).
- No chart re-rendering for historical KPs — only KPs captured in the current turn render charts; pre-Phase-2 KPs aren't back-filled.
- No "reset case" admin path — `/rewind` wipes session state but chart files persist on disk for audit.

---

10. Files of interest

concern	file
Session state, warmth, pruning	`server.py` (`CaseSession`, `_prune_input_history`, `_format_kb_warmth_hint`)
KB read path + distiller scheduling	`agent_factories/redacting_tool.py`
Per-turn context plumbing	`agent_factories/app_context.py`
Distiller agent + prompt	`agent_factories/distiller_agent.py`
`KnowledgePoint` schema	`models/types.py`
Distiller construction	`orchestrator/orchestrator.py`
Chart rendering	`tools/viz_renderer.py`
Routing skill (consumes warmth)	`skills/workflow/team_construction.md`
Original PRD	`tasks/prd-memory-management.md`

Appendix A — Specialist knowledge distillation, worked examples

Each example shows the same end-to-end shape: the **`SpecialistOutput`** the distiller is handed (left), the **`KnowledgePoint`** list it emits (right), and a short note on which prompt rules drove the transformation. The examples are synthesized to be self-contained — real payloads contain more boilerplate, but the distillation logic is identical.

A.1 Canonical case — single-series monthly trend

Specialist: `spend_payments` Sub-question: "What's the customer's spending pattern over the past 6 months?"

SpecialistOutput (input to distiller):


```
{
  "answer": "Spend rose sharply across the window. Monthly total grew from $300 in 2024-11 to a
  "findings": [
    "summarize_trend('spends','Amount','Date',period='month',op='sum') → 2024-11: 300, 2024-12:
    "The peak month (2025-03) is the highest in the available 6-month window."
  ],
  "data_gaps": [],
  "confidence": "high"
}
```


KnowledgePoint emitted (1):


```
{
  "topic": "monthly_spend_trend",
  "claim": "Spend rose from $300 (2024-11) to $1,100 (2025-03), a 3.7× increase peaking in 2025-
  "numbers": [
    {"period": "2024-11", "value": 300},
    {"period": "2024-12", "value": 250},
    {"period": "2025-01", "value": 480},
    {"period": "2025-02", "value": 920},
    {"period": "2025-03", "value": 1100},
    {"period": "2025-04", "value": 760}
  ],
  "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
  "source_call": "summarize_trend('spends','Amount','Date',period='month',op='sum')",
  "captured_at_turn": "a3f7c1e9d2b4",
  "confidence": "high"
}
```

Rules applied:

- Atomic: a 6-month series becomes one KP, with the series in `numbers`, not six separate KPs.
- Time window in claim matches first/last *`numbers`*: claim says "2024-11 … 2025-03"; series begins at 2024-11 and the cited peak month is 2025-03.
- Quantitative bias: only the magnitudes and the 3.7× multiplier are surfaced; the qualitative "softened" tail isn't promoted to a separate KP.
- *`viz`* set because `numbers` has ≥ 4 points and they share a periodic x-axis → renderer writes `reports/<case>/charts/a3f7c1e9d2b4-monthly_spend_trend.png`.

A.2 Two same-scale scores — the load-bearing per-metric topic rule

Specialist: `modeling` Sub-question: "How have the internal model scores moved over the past year?"

SpecialistOutput (input):

```
{
  "answer": "Both internal scores deteriorated through the window. CDSS (credit_loss_prob) climb
  "findings": [
    "summarize_trend('modelling_data','credit_loss_prob','trans_month',period='month',op='last')
    "summarize_trend('modelling_data','tot_struct_risk_score','trans_month',period='month',op='l
    "Catalog thresholds: credit_loss_prob risky > 0.5; tot_struct_risk_score risky > 20."
  ],
  "data_gaps": [],
  "confidence": "high"
}
```

Preferred shape — ONE multi-series KP:

```
{
  "topic": "internal_model_scores_trend",
  "claim": "CDSS rose from 0.12 to 0.61 (crossing the 0.5 risky threshold in 2025-03); TSR rose
  "numbers": [
    {"period": "2024-11", "credit_loss_prob": 0.12, "tot_struct_risk_score": 22.0,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20},
    {"period": "2024-12", "credit_loss_prob": 0.18, "tot_struct_risk_score": 23.4,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20},
    {"period": "2025-01", "credit_loss_prob": 0.31, "tot_struct_risk_score": 24.8,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20},
    {"period": "2025-02", "credit_loss_prob": 0.44, "tot_struct_risk_score": 26.1,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20},
    {"period": "2025-03", "credit_loss_prob": 0.55, "tot_struct_risk_score": 27.3,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20},
    {"period": "2025-04", "credit_loss_prob": 0.61, "tot_struct_risk_score": 28.5,
     "threshold_credit_loss_prob": 0.5, "threshold_tot_struct_risk_score": 20}
  ],
  "viz": {"kind": "trend_dual", "x_field": "period",
          "y_fields": ["credit_loss_prob", "tot_struct_risk_score"]},
  "source_call": "summarize_trend('modelling_data', ...) × 2",
  "captured_at_turn": "c8a3f2e5d917",
  "confidence": "high"
}
```

Acceptable fallback — TWO single-series KPs with per-metric topics:

```
[
  {"topic": "cdss_score_trend",
```

```
    "claim": "CDSS (credit_loss_prob) rose from 0.12 (2024-11) to 0.61 (2025-04), crossing the 0.
    "numbers": [
      {"period": "2024-11", "value": 0.12, "threshold": 0.5},
      ... (six rows, each with threshold=0.5) ...
    ],
    "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
    "source_call": "summarize_trend('modelling_data','credit_loss_prob',...)",
    ...},
  {"topic": "tsr_score_trend",
    "claim": "TSR (tot_struct_risk_score) rose from 22.0 (2024-11) to 28.5 (2025-04), breaching t
    "numbers": [
      {"period": "2024-11", "value": 22.0, "threshold": 20},
      ... (six rows, each with threshold=20) ...
    ],
    "viz": {"kind": "trend", "x_field": "period", "y_fields": ["value"]},
    "source_call": "summarize_trend('modelling_data','tot_struct_risk_score',...)",
    ...}
]
```

Rules applied — and the trap to avoid:

- Per-metric topic slugs. Using a generic family slug like `model_scores_trend` for both would collide: the second KP's chart would silently overwrite the first's PNG (chart collection dedupes by `topic`). The required forms are either one multi-series KP with a single topic, or two KPs with metric-specific topics (`cdss_score_trend`, `tsr_score_trend`).
- Thresholds on every row. `get_table_schema` documented "risky > 0.5" and "risky > 20", so each `numbers` row carries the threshold. The renderer draws a dashed reference line per axis. In the multi-series case the keys are `threshold_<y_field>` so each y-axis gets its own line.
- Claim faithfully cites the breach turns — both crossover periods named, both end-points cited.

A.3 Concentration / breakdown — **`numbers`** carries **`group`**, no chart

Specialist: `spend_payments` Sub-question: "Which merchants drive recurring spend?"

SpecialistOutput (input):

```
{
  "answer": "S BERTRAM is the dominant recurring merchant, taking 38% of total recurring spend (
  "findings": [
    "aggregate_column('spends','Merchant','Amount',op='sum') filtered to recurring transactions
  ],
  "data_gaps": [],
  "confidence": "medium"
}
```

KnowledgePoint emitted (1):

```
{
  "topic": "top_merchants_by_sum",
  "claim": "S BERTRAM accounts for 38% of recurring spend ($642K of $1.69M total); next three me
  "numbers": [
    {"group": "S BERTRAM",     "value": 642000},
    {"group": "MERCHANT_B",    "value": 198000},
    {"group": "MERCHANT_C",    "value": 142000},
    {"group": "MERCHANT_D",    "value": 86000},
    {"group": "OTHER (n=45)",  "value": 622000}
  ],
```

```
  "viz": null,
  "source_call": "aggregate_column('spends','Merchant','Amount',op='sum')",
  "captured_at_turn": "a3f7c1e9d2b4",
  "confidence": "medium"
}
```

Rules applied:

- Breakdown shape: `numbers` uses `{group, value}` rather than `{period, value}` — the renderer adapts when `viz` is supplied, but here we leave it `null` because the share-of-total framing is better read as the prose claim than as a small bar chart.
- Confidence carried through from the specialist ("medium" because the source merchant strings can drift across months).
- Named entity in *`claim`* (`S BERTRAM`, the dominant merchant) is what makes this point reusable on follow-up turns — future questions like "is S BERTRAM still the top driver?" can be answered from the KB without re-aggregating.

A.4 What the distiller drops on the floor

For the same `spend_payments` turn, suppose the answer text also contains:

> "There is no payment-returns data prior to 2024-11, so we can't compare against 2024-Q3. Spending appears volatile, which may indicate seasonal influence — though without more history it's hard to tell."

Nothing in the above promotes to a KP:

sentence	dropped because
"no payment-returns data prior to 2024-11 … can't compare"	Data-absence. Already lives in `SpecialistOutput.data_gaps`; the rule says "skip absence-of-data".
"Spending appears volatile, which may indicate seasonal influence"	Pure narrative — no numbers, no named entity, no comparison the next turn could fact-check.
"though without more history it's hard to tell"	Hedge-only, not a quantitative claim.

This is why distillation reliably produces a small KB even when the specialist's prose is long: the rules privilege re-usable, fact-checkable numerics.

A.5 How the KPs feed back into the next turn

Two turns later, the reviewer asks "Has the spending peak persisted?" and the orchestrator (cued by the warmth hint) re-routes to `spend_payments`. The first call within the new turn includes the KB digest:

```
[YOUR KNOWLEDGE BASE — facts established earlier this session.
 Refer to these BEFORE re-running queries; only re-query when the new
 question goes beyond what's recorded here, or when a value needs verification.]

- **monthly_spend_trend** [high]: Spend rose from $300 (2024-11) to $1,100 (2025-
03), a 3.7× increase peaking in 2025-Q1. _via `summarize_trend('spends','Amount','Date',period='mo
- **top_merchants_by_sum** [medium]: S BERTRAM accounts for 38% of recurring spend ($642K of $1.69M t
- **internal_model_scores_trend** [high]: CDSS rose from 0.12 to 0.61 (crossing 0.5 in 2025-
03); TSR rose from 22.0 to 28.5, breaching 20 from 2024-12 onward. _via `summarize_trend('modelling_
```

```
--- New question ---
Has the spending peak persisted?
```

The specialist now knows — without an extra tool call — that the peak it should be testing for persistence is 2025-03 at $1,100. Its only new query is the one that extends or refutes that claim. If the new data does refute the prior trend (say the peak has shifted), the distiller's next pass emits a fresh `monthly_spend_trend` KP and the older one is shadowed in the active view but kept in the append-only list for audit.

This is the loop the whole subsystem exists to make tight: distill once, reuse cheaply, never silently lose what was believed.